

Structure of Programming Language

Using Python

Data types in Python

Number, List and Tuple

Objectives

At the end of the module the student is expected to

List the different data types in Python

Define the following

List

Tuple

String

Set

Dictionary

Apply concepts using python language.

Topics

Data Types

Numbers

List

Tuple

String

Set

Dictionary

Number Data Type in Python

Python supports integers, floating-point numbers and complex numbers. They are defined as int, float, and complex classes in Python.

Integers and floating points are separated by the presence or absence of a decimal point. For instance, 5 is an integer whereas 5.0 is a floating-point number.

Complex numbers are written in the form, $x + yj$, where x is the real part and y is the imaginary part.

We can use the `type()` function to know which class a variable or a value belongs to and `isinstance()` function to check if it belongs to a particular class.

Example

```
1 a = 5
2
3 print(type(a))
4
5 print(type(5.0))
6
7 c = 5 + 3j
8 print(c + 3)
9
10 print(isinstance(c, complex))|
```

Program Code

```
<class 'int'>
<class 'float'>
(8+3j)
True
```

Program Output

While integers can be of any length, a floating-point number is accurate only up to 15 decimal places (the 16th place is inaccurate).

The numbers we deal with every day are of the decimal (base 10) number system. But computer programmers (generally embedded programmers) need to work with binary (base 2), hexadecimal (base 16) and octal (base 8) number systems.

Number System

```
1 # Output: 107
2 print(0b1101011)
3
4 # Output: 253 (251 + 2)
5 print(0xFB + 0b10)
6
7 # Output: 13
8 print(0o15)
```

Program Code

107
253
13

Program Output

Number System	Prefix
Binary	'0b' or '0B'
Octal	'0o' or '0O'
Hexadecimal	'0x' or '0X'

Type Conversion

We can convert one type of number into another. This is also known as coercion.

Operations like addition, subtraction coerce integer to float implicitly (automatically), if one of the operands is float.

```
>>> 1 + 2.0  
3.0
```

We can see above that 1 (integer) is coerced into 1.0 (float) for addition and the result is also a floating point number.

We can also use built-in functions like **int()**, **float()** and **complex()** to convert between types explicitly. These functions can even convert from strings.

```
1 print(int(2.3))
2
3 print(int(-2.8))
4
5 print(float(5))
6
7 print(complex('3+5j'))
8
```

Program Code

```
2
-2
5.0
(3+5j)
```

Program Output

When converting from float to integer, the number gets truncated (decimal parts are removed).

Python Decimal

Python built-in class float performs some calculations that might amaze us. We all know that the sum of 1.1 and 2.2 is 3.3, but Python seems to disagree.

```
1 print((1.1 + 2.2)==3.3)

False
```

What is going on?

It turns out that floating-point numbers are implemented in computer hardware as binary fractions as the computer only understands binary (0 and 1). Due to this reason, most of the decimal fractions we know, cannot be accurately stored in our computer.

Let's take an example. We cannot represent the fraction $1/3$ as a decimal number. This will give 0.3333333... which is infinitely long, and we can only approximate it.

It turns out that the decimal fraction 0.1 will result in an infinitely long binary fraction of 0.000110011001100110011... and our computer only stores a finite number of it.

This will only approximate 0.1 but never be equal. Hence, it is the limitation of our computer hardware and not an error in Python.

When to use Decimal instead of float?

We generally use Decimal in the following cases.

1. When we are making financial applications that need exact decimal representation.
2. When we want to control the level of precision required.
3. When we want to implement the notion of significant decimal places.

Python Fractions

Python provides operations involving fractional numbers through its fractions module.

A fraction has a numerator and a denominator, both of which are integers. This module has support for rational number arithmetic.

```

1 import fractions
2
3 print(fractions.Fraction(1.5))
4
5 print(fractions.Fraction(5))
6
7 print(fractions.Fraction(1,3))

```

```

3/2
5
1/3

```

While creating Fraction from float, we might get some unusual results. This is due to the imperfect binary floating point number representation as discussed in the previous section.

Fortunately, Fraction allows us to instantiate with string as well. This is the preferred option when using decimal numbers.

```

1 import fractions
2
3 # As float
4 # Output: 2476979795053773/2251799813685248
5 print(fractions.Fraction(1.1))
6
7 # As string
8 # Output: 11/10
9 print(fractions.Fraction('1.1'))

```

```

2476979795053773/2251799813685248
11/10

```



PEU ALABANG



PEU SULAIMAN



PEU TECH

Technology Driven by Innovation

Basic operations

```
1 from fractions import Fraction as F
2
3 print(F(1, 3) + F(1, 3))
4
5 print(1 / F(5, 6))
6
7 print(F(-3, 10) > 0)
8
9 print(F(-3, 10) < 0)
```

2/3

6/5

False

True

Python Mathematics

Python offers modules like math and random to carry out different mathematics like trigonometry, logarithms, probability and statistics, etc.

```
1 import math
2
3 print(math.pi)
4 print(math.cos(math.pi))
5 print(math.exp(10))
6 print(math.log10(1000))
7 print(math.sinh(1))
8 print(math.factorial(6))
```

```
3.141592653589793
-1.0
22026.465794806718
3.0
1.1752011936438014
720
```


Example

```
1 import random
2
3 print(random.randrange(10, 20))
4 x = ['a', 'b', 'c', 'd', 'e']
5 # Get random choice
6 print(random.choice(x))
7 # Shuffle x
8 random.shuffle(x)
9 # Print the shuffled x
10 print(x)
11 # Print random element
12 print(random.random())
```

```
13
d
['c', 'a', 'e', 'b', 'd']
0.178828363540816
```

When we run the above program we get the output as follows.

(Values may be different due to the random behavior)



Python List

Python List

A **list** is a data structure in Python that is a mutable, or changeable, ordered sequence of elements. Each element or value that is inside of a list is called an item. Just as strings are defined as characters between quotes, lists are defined by having values between square brackets [].

Lists are great to use when you want to work with many related values. They enable you to keep data together that belongs together, condense your code, and perform the same methods and operations on multiple values at once.

How to create a list?

In Python programming, a list is created by placing all the items (elements) inside square brackets [], separated by commas.

It can have any number of items and they may be of different types (integer, float, string etc.).

```
# empty list my_list = []  
# list of integers my_list = [1, 2, 3]  
# list with mixed data types  
my_list = [1, "Hello", 3.4]
```

A list can also have another list as an item. This is called a nested list.

```
# nested list my_list = ["mouse", [8, 4, 6], ['a']]
```

How to access elements from a list?

List Index

We can use the index operator `[]` to access an item in a list. In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to 4.

Trying to access indexes other than these will raise an `IndexError`. The index must be an integer. We can't use float or other types, this will result in `TypeError`.

```

1 # List indexing
2 my_list = ['H', 'a', 'p', 'p', 'y']
3 # Output: H
4 print(my_list[0])
5 # Output: p
6 print(my_list[2])
7 # Output: y
8 print(my_list[4])
9 # Nested List
10 n_list = ["Happy", [2, 0, 1, 5]]
11 # Nested indexing
12 print(n_list[0][1])
13 print(n_list[1][3])
14 # Error! Only integer can be used for indexing
15 print(my_list[4.0])

```

H
p
y
a
5

Traceback (most recent call last):

File "Untitled 7.py", line 22, in <module>
print(my_list[4.0])

TypeError: list indices must be integers or slices, not float

Negative indexing

Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on.

```
1 # Negative indexing in lists
2 my_list = ['h', 'a', 'p', 'p', 'y']
3
4 print(my_list[-1])
5
6 print(my_list[-5])
```

y
h

How to slice lists in Python?

We can access a range of items in a list by using the slicing operator `:`(colon).

```
1 # List slicing in Python
2
3 my_list = ['h','a','d','j','i','p','o','g','i']
4 # elements 3rd to 5th
5 print(my_list[2:5])
6 # elements beginning to 4th
7 print(my_list[:5])
8 # elements 6th to end
9 print(my_list[5:])
10 # elements beginning to end
11 print(my_list[:])
```



```
['h', 'a', 'd', 'j', 'i', 'p', 'o', 'g', 'i']
['d', 'j', 'i']
['h', 'a', 'd', 'j']
['p', 'o', 'g', 'i']
['h', 'a', 'd', 'j', 'i', 'p', 'o', 'g', 'i']
```


How to change or add elements to a list?

```
1 # Correcting mistake values in a list
2 odd = [2, 4, 6, 8]
3 # change the 1st item
4 odd[0] = 1
5 print(odd)
6 # change 2nd to 4th items
7 odd[1:4] = [3, 5, 7]
8 print(odd)
```

```
[1, 4, 6, 8]
[1, 3, 5, 7]
```

Lists are mutable, meaning their elements can be changed unlike [string](#) or [tuple](#).

We can use the assignment operator (=) to change an item or a range of items.

Appending and Extending lists in Python

```
1 # Appending and Extending lists in Python
2 odd = [1, 3, 5]
3 odd.append(7)
4 print(odd)
5 odd.extend([9, 11, 13])
6 print(odd)
```

```
[1, 3, 5, 7]
[1, 3, 5, 7, 9, 11, 13]
```

We can add one item to a list using the `append()` method or add several items using `extend()` method.

Concatenating and repeating lists

```
1 # Concatenating and repeating lists
2 odd = [1, 3, 5]
3
4 print(odd + [9, 7, 5])
5
6 print(["re"] * 3)
```

```
[1, 3, 5, 9, 7, 5]
['re', 're', 're']
```

We can also use + operator to combine two lists. This is also called concatenation.

The * operator repeats a list for the given number of times.

Demonstration of list insert() method

```
1 # Demonstration of list insert() method
2 odd = [1, 9]
3 odd.insert(1,3)
4 print(odd)
5 odd[2:2] = [5, 7]
6 print(odd)
```

```
[1, 3, 9]
```

```
[1, 3, 5, 7, 9]
```

Furthermore, we can insert one item at a desired location by using the method insert() or insert multiple items by squeezing it into an empty slice of a list.

How to delete or remove elements from a list?

```
1 # Deleting list items
2 my_list = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
3 # delete one item
4 del my_list[2]
5 print(my_list)
6 # delete multiple items
7 del my_list[1:5]
8 print(my_list)
9 # delete entire list
10 del my_list
11 # Error: List not defined
12 print(my_list)
```

```
['a', 'b', 'd', 'e', 'f', 'g']
['a', 'g']
```

Traceback (most recent call last):

```
File "Untitled 7.py", line 12, in <module>
    print(my_list)
```

NameError: name 'my_list' is not defined

We can delete one or more items from a list using the keyword `del`. It can even delete the list entirely.

```

1 my_list = ['p','r','o','b','l','e','m']
2 my_list.remove('p')
3 # Output: ['r', 'o', 'b', 'l', 'e', 'm']
4 print(my_list)
5 # Output: 'o'
6 print(my_list.pop(1))
7 # Output: ['r', 'b', 'l', 'e', 'm']
8 print(my_list)
9 # Output: 'm'
10 print(my_list.pop())
11 # Output: ['r', 'b', 'l', 'e']
12 print(my_list)
13 my_list.clear()
14 # Output: []
15 print(my_list)

```

```

['r', 'o', 'b', 'l', 'e', 'm']
o
['r', 'b', 'l', 'e', 'm']
m
['r', 'b', 'l', 'e']
[]

```

We can use `remove()` method to remove the given item or `pop()` method to remove an item at the given index.

The `pop()` method removes and returns the last item if the index is not provided. This helps us implement lists as stacks (first in, last out data structure).

We can also use the `clear()` method to empty a list.



Python List Methods

Python List Methods	
append()	- Add an element to the end of the list
extend()	- Add all elements of a list to the another list
insert()	- Insert an item at the defined index
remove()	- Removes an item from the list
pop()	- Removes and returns an element at the given index
clear()	- Removes all items from the list
index()	- Returns the index of the first matched item
count()	- Returns the count of the number of items passed as an argument
sort()	- Sort items in a list in ascending order
reverse()	- Reverse the order of items in the list
copy()	- Returns a shallow copy of the list

Methods that are available with list objects in Python programming are tabulated below.

They are accessed as `list.method()`. Some of the methods have already been used above.

Examples

```
1 # Python list methods
2 my_list = [3, 8, 1, 6, 0, 8, 4]
3 # Output: 1
4 print(my_list.index(8))
5 # Output: 2
6 print(my_list.count(8))
7 my_list.sort()
8 # Output: [0, 1, 3, 4, 6, 8, 8]
9 print(my_list)
10 my_list.reverse()
11 # Output: [8, 8, 6, 4, 3, 1, 0]
12 print(my_list)
```

```
1
2
[0, 1, 3, 4, 6, 8, 8]
[8, 8, 6, 4, 3, 1, 0]
```


List Comprehension: Elegant way to create new List

List comprehension is an elegant and concise way to create a new list from an existing list in Python.

A list comprehension consists of an expression followed by for statement inside square brackets.

```
1 pow2 = [2 ** x for x in range(10)]
2 print(pow2)
```



```
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

Here is an example to make a list with each item being increasing power of 2.

This code is equivalent to:

```
1 pow2 = []
2 for x in range(10):
3     pow2.append(2 ** x)
4 print(pow2)
```



```
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

Other List Operations in Python

List Membership Test

We can test if an item exists in a list or not, using the keyword `in`.

```
1 my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']
2 # Output: True
3 print('p' in my_list)
4 # Output: False
5 print('a' in my_list)
6 # Output: True
7 print('c' not in my_list)
```



```
True
False
True
```

Iterating Through a List

Using a for loop we can iterate through each item in a list.

```
1  for fruit in ['apple', 'banana', 'mango']:  
2      print("I like", fruit)  
  
I like apple  
I like banana  
I like mango
```

Python Tuple

Python Tuple

A tuple in Python is similar to a list. The difference between the two is that we cannot change the elements of a tuple once it is assigned whereas we can change the elements of a list.

A tuple is a data structure that is an immutable, or unchangeable, ordered sequence of elements. Because tuples are immutable, their values cannot be modified.

Creating a Tuple

A tuple is created by placing all the items (elements) inside parentheses (), separated by commas. The parentheses are optional, however, it is a good practice to use them.

A tuple can have any number of items and they may be of different types (integer, float, list, [string](#), etc.).

Different types of tuples

```
1 # Different types of tuples
2
3 # Empty tuple
4 my_tuple = ()
5 print(my_tuple)
6 # Tuple having integers
7 my_tuple = (1, 2, 3)
8 print(my_tuple)
9 # tuple with mixed datatypes
10 my_tuple = (1, "Hello", 3.4)
11 print(my_tuple)
12 # nested tuple
13 my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
14 print(my_tuple)
```

```
()
(1, 2, 3)
(1, 'Hello', 3.4)
('mouse', [8, 4, 6], (1, 2, 3))
```

A tuple can also be created without using parentheses. This is known as tuple packing.

```
1 my_tuple = 3, 4.6, "dog"
2 print(my_tuple)
3
4 # tuple unpacking is also possible
5 a, b, c = my_tuple
6
7 print(a)      # 3
8 print(b)      # 4.6
9 print(c)      # dog
```

```
(3, 4.6, 'dog')
```

```
3
```

```
4.6
```

```
dog
```

Access Tuple Elements

Indexing

We can use the index operator `[]` to access an item in a tuple, where the index starts from 0.

So, a tuple having 6 elements will have indices from 0 to 5. Trying to access an index outside of the tuple index range(6,7,... in this example) will raise an `IndexError`.

The index must be an integer, so we cannot use float or other types. This will result in `TypeError`.

Accessing tuple elements using indexing

```
1 # Accessing tuple elements using indexing
2 my_tuple = ('p','e','r','m','i','t')
3
4 print(my_tuple[0])    # 'p'
5 print(my_tuple[5])    # 't'
6
7 # IndexError: list index out of range
8 # print(my_tuple[6])
9
10 # Index must be an integer
11 # TypeError: list indices must be integers, not float
12 # my_tuple[2.0]
13
14 # nested tuple
15 n_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
16
17 # nested index
18 print(n_tuple[0][3])    # 's'
19 print(n_tuple[1][1])    # 4
```

p
t
s
4



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Access Tuple Elements

Negative Indexing

Python allows negative indexing for its sequences.

The index of -1 refers to the last item, -2 to the second last item and so on.

```
1 # Negative indexing for accessing tuple elements
2 my_tuple = ('p', 'e', 'r', 'm', 'i', 't')
3
4 # Output: 't'
5 print(my_tuple[-1])
6
7 # Output: 'p'
8 print(my_tuple[-6])
```

t

p

Slicing

We can access a range of items in a tuple by using the slicing operator colon :.

```
1 # Accessing tuple elements using slicing
2 my_tuple = ('h','a','d','j','i','p','o','g','i')
3 # elements 2nd to 4th
4 print(my_tuple[1:4])
5 # elements beginning to 2nd
6 print(my_tuple[:2])
7 # elements 8th to end
8 print(my_tuple[7:])
9 # elements beginning to end
10 print(my_tuple[:])
```



```
('a', 'd', 'j')
('h', 'a')
('g', 'i')
('h', 'a', 'd', 'j', 'i', 'p', 'o', 'g', 'i')
```

Changing a Tuple

Unlike lists, tuples are immutable.

This means that elements of a tuple cannot be changed once they have been assigned. But, if the element is itself a mutable data type like list, its nested items can be changed.

Changing tuple values

```
1 # Changing tuple values
2 my_tuple = (4, 2, 3, [6, 5])
3
4 # TypeError: 'tuple' object does not support item assignment
5 # my_tuple[1] = 9
6
7 # However, item of mutable element can be changed
8 my_tuple[3][0] = 9 # Output: (4, 2, 3, [9, 5])
9 print(my_tuple)
10
11 # Tuples can be reassigned
12 my_tuple = ('h', 'a', 'd', 'j', 'i', 'p', 'o', 'g', 'i')
13
14 # Output: ('h', 'a', 'd', 'j', 'i', 'p', 'o', 'g', 'i')
15 print(my_tuple)|
```

(4, 2, 3, [9, 5])

('h', 'a', 'd', 'j', 'i', 'p', 'o', 'g', 'i')

We can use + operator to combine two tuples. This is called **concatenation**.

We can also **repeat** the elements in a tuple for a given number of times using the * operator.

Both + and * operations result in a new tuple.

Concatenation

```
1 # Concatenation
2 # Output: (1, 2, 3, 4, 5, 6)
3 print((1, 2, 3) + (4, 5, 6))
4
5 # Repeat
6 # Output: ('Repeat', 'Repeat', 'Repeat')
7 print(("Repeat",) * 3)
```

```
(1, 2, 3, 4, 5, 6)
('Repeat', 'Repeat', 'Repeat')
```

Deleting a Tuple

Deleting a tuple entirely, however, is possible using the keyword `del`.

```
1 # Deleting tuples
2 my_tuple = ('a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i')
3
4 # can't delete items
5 # TypeError: 'tuple' object doesn't support item deletion
6 # del my_tuple[3]
7
8 # Can delete an entire tuple
9 del my_tuple
10
11 # NameError: name 'my_tuple' is not defined
12 print(my_tuple)
```

```
Traceback (most recent call last):
  File "Untitled 7.py", line 12, in <module>
    print(my_tuple)
NameError: name 'my_tuple' is not defined
```

Tuple Methods

Methods that add items or remove items are not available with tuple. Only the following two methods are available.

Some examples of Python tuple methods:

```
1 my_tuple = ('a', 'p', 'p', 'l', 'e',)
2
3 print(my_tuple.count('p')) # Output: 2
4 print(my_tuple.index('l')) # Output: 3
```



```
2
3
```

Other Tuple Operation

Tuple Membership Test

We can test if an item exists in a tuple or not, using the keyword `in`.

```
1 # Membership test in tuple
2 my_tuple = ('a', 'p', 'p', 'l', 'e',)
3
4 # In operation
5 print('a' in my_tuple)
6 print('b' in my_tuple)
7
8 # Not in operation
9 print('g' not in my_tuple)
```

```
True
False
True
```

Other Tuple Operation

Iterating Through a Tuple

We can use a for loop to iterate through each item in a tuple.

```
1 # Using a for loop to iterate through a tuple
2 for name in ('Hadji', 'Joan'):
3     print("Hello", name)
```



```
Hello Hadji
Hello Joan
```


Advantages of Tuple over List

Since tuples are quite similar to lists, both of them are used in similar situations. However, there are certain advantages of implementing a tuple over a list. Below listed are some of the main advantages:

1. We generally use tuples for heterogeneous (different) data types and lists for homogeneous (similar) data types.
2. Since tuples are immutable, iterating through a tuple is faster than with list. So there is a slight performance boost.
3. Tuples that contain immutable elements can be used as a key for a dictionary. With lists, this is not possible.
4. If you have data that doesn't change, implementing it as tuple will guarantee that it remains write-protected.

Data types in Python

String, Set and Dictionary

Objectives

At the end of the module the student is expected to

List the different data types in Python

Define the following

List

Tuple

String

Set

Dictionary

Apply concepts using python language.

Topics

Data Types

Numbers

List

Tuple

String

Set

Dictionary

What is String in Python?

A string is a sequence of characters.

A character is simply a symbol. For example, the English language has 26 characters.

Computers do not deal with characters, they deal with numbers (binary). Even though you may see characters on your screen, internally it is stored and manipulated as a combination of 0s and 1s.

This conversion of character to a number is called encoding, and the reverse process is decoding. ASCII and Unicode are some of the popular encodings used

How to create a string in Python?

Strings can be created by enclosing characters inside a single quote or double-quotes. Even triple quotes can be used in Python but generally used to represent multiline strings and docstrings.

```
1 # defining strings in Python
2 # all of the following are equivalent
3 my_string = 'Hello'
4 print(my_string)
5
6 my_string = "Hello"
7 print(my_string)
8
9 my_string = '''Hello'''
10 print(my_string)
11
12 # triple quotes string can extend multiple lines
13 my_string = """Hello, welcome to
14     .....the world of Python"""
15 print(my_string)
```

```
Hello
Hello
Hello
Hello, welcome to
                        the world of Python
```

How to access characters in a string?

We can access individual characters using indexing and a range of characters using slicing. Index starts from 0. Trying to access a character out of index range will raise an `IndexError`. The index must be an integer. We can't use floats or other types, this will result into `TypeError`.

Python allows negative indexing for its sequences.

The index of -1 refers to the last item, -2 to the second last item and so on. We can access a range of items in a string by using the slicing operator `:(colon)`.

Accessing string characters in Python

```
1 #Accessing string characters in Python
2 str = 'FEUTECH'
3 print('str = ', str)
4
5 #first character
6 print('str[0] = ', str[0])
7
8 #last character
9 print('str[-1] = ', str[-1])
10
11 #slicing 2nd to 5th character
12 print('str[1:5] = ', str[1:5])
13
14 #slicing 6th to 2nd last character
15 print('str[5:-2] = ', str[5:-2])
```

```
str = FEUTECH
str[0] = F
str[-1] = H
str[1:5] = EUTE
str[5:-2] =
```

How to change or delete a string?

Strings are immutable. This means that elements of a string cannot be changed once they have been assigned. We can simply reassign different strings to the same name.

```
1 my_string = 'FEUTECH'
2 my_string[5] = 'a'
3
4 my_string = 'Python'
5 print(my_string)
6
7 del my_string[1]
```

Traceback (most recent call last):
File "Untitled 7.py", line 2, in <module>
my_string[5] = 'a'
TypeError: 'str' object does not support item assignment

We cannot delete or remove characters from a string. But deleting the string entirely is possible using the del keyword.

Python String Operations

There are many operations that can be performed with string which makes it one of the most used data types in Python.

Concatenation of Two or More Strings

Joining of two or more strings into a single one is called concatenation.

The `+` operator does this in Python. Simply writing two string literals together also concatenates them.

The `*` operator can be used to repeat the string for a given number of times.

Python String Operations

```
1 # Python String Operations
2 str1 = 'Hello'
3 str2 = 'World!'
4
5 # using +
6 print('str1 + str2 = ', str1 + str2)
7
8 # using *
9 print('str1 * 3 =', str1 * 3)
```

Writing two string literals together also concatenates them like + operator.

```
str1 + str2 = HelloWorld!
str1 * 3 = HelloHelloHello
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

two string literals together

```
1 # two string literals together
2 print('Hello ' 'World!')
3
4 # using parentheses
5 s = ('Hello '
6     ....'World')
7 print(s)
8
```

```
Hello World!
Hello World
```

If we want to concatenate strings in different lines, we can use parentheses.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Iterating Through a string

```
1 # Iterating through a string
2 count = 0
3 for letter in 'Hello World':
4     if(letter == 'l'):
5         count += 1
6 print(count, 'letters found')
```

3 letters found

Built-in functions to Work with Python

Various built-in functions that work with sequence work with strings as well.

Some of the commonly used ones are `enumerate()` and `len()`.

The `enumerate()` function returns an enumerate object. It contains the index and value of all the items in the string as pairs. This can be useful for iteration.

Similarly, `len()` returns the length (number of characters) of the string.

Example

```
1 str = 'hadji'
2
3 # enumerate()
4 list_enumerate = list(enumerate(str))
5 print('list(enumerate(str) = ', list_enumerate)
6
7 #character count
8 print('len(str) = ', len(str))
```

```
list(enumerate(str) = [(0, 'h'), (1, 'a'), (2, 'd'), (3, 'j'), (4, 'i')]
len(str) = 5
```



Python String Formatting

If we want to print a text like He said, "What's there?", we can neither use single quotes nor double quotes. This will result in a `SyntaxError` as the text itself contains both single and double quotes.

```
1 print("He said, "What's there?")
2
3 print('He said, "What's there?')
4
```



```
File "Untitled 7.py", line 1
    print("He said, "What's there?")
                    ^
SyntaxError: invalid syntax
```

Escape sequence

One way to get around this problem is to use triple quotes.

Alternatively, we can use escape sequences.

An escape sequence starts with a backslash and is interpreted differently. If we use a single quote to represent a string, all the single quotes inside the string must be escaped. Similar is the case with double quotes. Here is how it can be done to represent the above text.

```
1 # using triple quotes
2 print(''He said, "What's there?''')
3
4 # escaping single quotes
5 print('He said, "What\'s there?")
6
7 # escaping double quotes
8 print("He said, \"What's there?\")
```

```
He said, "What's there?"
He said, "What's there?"
He said, "What's there?"
```

Escape Sequence

Escape Sequence	Description
<code>\newline</code>	Backslash and newline ignored
<code>\\</code>	Backslash
<code>\'</code>	Single quote
<code>\"</code>	Double quote
<code>\a</code>	ASCII Bell
<code>\b</code>	ASCII Backspace
<code>\f</code>	ASCII Formfeed
<code>\n</code>	ASCII Linefeed
<code>\r</code>	ASCII Carriage Return
<code>\t</code>	ASCII Horizontal Tab
<code>\v</code>	ASCII Vertical Tab
<code>\ooo</code>	Character with octal value ooo
<code>\xHH</code>	Character with hexadecimal value HH

```
1 print("C:\\Python32\\Lib")
2
3 print("This is printed\nin two lines")
4
5 print("This is \x48\x45\x58 representation")
```

C:\Python32\Lib
This is printed
in two lines
This is HEX representation

Technology Driven by Innovation

Raw String to ignore escape sequence

Sometimes we may wish to ignore the escape sequences inside a string. To do this we can place `r` or `R` in front of the string. This will imply that it is a raw string and any escape sequence inside it will be ignored.

```
>>> print("This is \x61 \ngood example")
```

```
This is a  
good example
```

```
>>> print(r"This is \x61 \ngood example")
```

```
This is \x61 \ngood example
```


The `format()` Method for Formatting Strings

The **`format()`** method that is available with the string object is very versatile and powerful in formatting strings. Format strings contain curly braces `{}` as placeholders or replacement fields which get replaced.

Python string format() method

```
1 # Python string format() method
2
3 # default(implicit) order
4 default_order = "{} , {} and {}".format('Hadji','Joan','Jeiven')
5 print('\n--- Default Order ---')
6 print(default_order)
7
8 # order using positional argument
9 positional_order = "{1} , {0} and {2}".format('Johann','Jeiven','Hadji')
10 print('\n--- Positional Order ---')
11 print(positional_order)
12
13 # order using keyword argument
14 keyword_order = "{s} , {b} and {j}".format(j='Joan',b='Johann',s='Jeiven')
15 print('\n--- Keyword Order ---')
16 print(keyword_order)
```

--- Default Order ---
Hadji, Joan and Jeiven

--- Positional Order ---
Jeiven, Johann and Hadji

--- Keyword Order ---
Jeiven, Johann and Joan

Technology Driven by Innovation

Format()

The format() method can have optional format specifications. They are separated from the field name using colon. For example, we can left-justify <, right-justify > or center ^ a string in the given space.

We can also format integers as binary, hexadecimal, etc. and floats can be rounded or displayed in the exponent format. There are tons of formatting you can use.

Examples

```
1 # formatting integers
2 print("Binary representation of {0} is {0:b}".format(12))
3
4 # formatting floats
5 print("Exponent representation: {0:e}".format(1566.345))
6
7 # round off
8 print("One third is: {0:.3f}".format(1/3))
9
10 # string alignment
11 print("|{:<10}|{: ^10}|{:>10}|".format('butter', 'bread', 'ham'))
```

Binary representation of 12 is 1100

Exponent representation: 1.566345e+03

One third is: 0.333

|butter | bread | ham|



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Old style formatting

```
1 x = 12.3456789
2 print('The value of x is %3.2f' %x)
3
4 print('The value of x is %3.4f' %x)
5
```

```
The value of x is 12.35
The value of x is 12.3457
```

We can even format strings like the old `sprintf()` style used in C programming language. We use the `%` operator to accomplish this.

Common Python String Methods

There are numerous methods available with the string object. The `format()` method that we mentioned above is one of them. Some of the commonly used methods are `lower()`, `upper()`, `join()`, `split()`, `find()`, `replace()` etc.

Example

```
1 print("HADJI TEJUCO".lower())
2
3 print("hadji tejuco".upper())
4
5 print("This will split all words into a list".split())
6 print(' '.join(['This', 'will', 'join', 'all', 'words', 'into', 'a', 'string']))
7
8 print('Happy New Year'.find('ew'))
9
10 print('Happy New Year'.replace('Happy', 'Brilliant'))|
```

hadji tejuco

HADJI TEJUCO

['This', 'will', 'split', 'all', 'words', 'into', 'a', 'list']

This will join all words into a string

7

Brilliant New Year



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Sets

Python Sets

A set is an unordered collection of items. Every set element is unique (no duplicates) and must be immutable (cannot be changed).

However, a set itself is mutable. We can add or remove items from it.

Sets can also be used to perform mathematical set operations like union, intersection, symmetric difference, etc.

Creating Python Sets

A set is created by placing all the items (elements) inside curly braces {}, separated by comma, or by using the built-in set() function.

It can have any number of items and they may be of different types (integer, float, tuple, string etc.). But a set cannot have mutable elements like lists, sets or dictionaries as its elements.

Example

```
1 # Different types of sets in Python
2 # set of integers
3 my_set = {1, 2, 3}
4 print(my_set)
5
6 # set of mixed datatypes
7 my_set = {1.0, "Hello", (1, 2, 3)}
8 print(my_set)
```

```
{1, 2, 3}
{'Hello', 1.0, (1, 2, 3)}
```

```
1 # set cannot have duplicates
2 # Output: {1, 2, 3, 4}
3 my_set = {1, 2, 3, 4, 3, 2}
4 print(my_set)
5
6 # we can make set from a list
7 # Output: {1, 2, 3}
8 my_set = set([1, 2, 3, 2])
9 print(my_set)
10
11 # set cannot have mutable items
12 # here [3, 4] is a mutable list
13 # this will cause an error.
14
15 my_set = {1, 2, [3, 4]}
```

```
{1, 2, 3, 4}
{1, 2, 3}
```

```
Traceback (most recent call last):
  File "Untitled 7.py", line 15, in <module>
    my_set = {1, 2, [3, 4]}
TypeError: unhashable type: 'list'
```

Creating an empty set is a bit tricky.

```
1 # Distinguish set and dictionary while creating empty set
2 # initialize a with {}
3 a = {}
4 # check data type of a
5 print(type(a))
6 # initialize a with set()
7 a = set()
8 # check data type of a
9 print(type(a))
```

```
<class 'dict'>
<class 'set'>
```

Empty curly braces {} will make an empty dictionary in Python. To make a set without any elements, we use the set() function without any argument.

Modifying a set in Python

Sets are mutable. However, since they are unordered, indexing has no meaning.

We cannot access or change an element of a set using indexing or slicing. Set data type does not support it.

We can add a single element using the `add()` method, and multiple elements using the `update()` method.

The `update()` method can take tuples, lists, strings or other sets as its argument. In all cases, duplicates are avoided.

```

1 # initialize my_set
2 my_set = {1, 3}
3 print(my_set)
4 # if you uncomment line 7,
5 # you will get an error
6 # TypeError: 'set' object does not support indexing
7 #my_set[0]
8 # add an element
9 # Output: {1, 2, 3}
10 my_set.add(2)
11 print(my_set)
12 # add multiple elements
13 # Output: {1, 2, 3, 4}
14 my_set.update([2, 3, 4])
15 print(my_set)
16 # add list and set
17 # Output: {1, 2, 3, 4, 5, 6, 8}
18 my_set.update([4, 5], {1, 6, 8})
19 print(my_set)

```

```

{1, 3}
{1, 2, 3}
{1, 2, 3, 4}
{1, 2, 3, 4, 5, 6, 8}

```


Removing elements from a set

A particular item can be removed from a set using the methods `discard()` and `remove()`.

The only difference between the two is that the `discard()` function leaves a set unchanged if the element is not present in the set. On the other hand, the `remove()` function will raise an error in such a condition (if element is not present in the set).

Example

```
1 # Difference between discard() and remove()
2 # initialize my_set
3 my_set = {1, 3, 4, 5, 6}
4 print(my_set)
5 # discard an element
6 # Output: {1, 3, 5, 6}
7 my_set.discard(4)
8 print(my_set)
9 # remove an element
10 # Output: {1, 3, 5}
11 my_set.remove(6)
12 print(my_set)
13 # discard an element
14 # not present in my_set
15 # Output: {1, 3, 5}
16 my_set.discard(2)
17 print(my_set)
18 # remove an element
19 # not present in my_set
20 # you will get an error.
21 # Output: KeyError
22 my_set.remove(2)
```

```
{1, 3, 4, 5, 6}
{1, 3, 5, 6}
{1, 3, 5}
{1, 3, 5}
```

Traceback (most recent call last):

File "Untitled 7.py", line 23, in <module>

my_set.remove(2)

KeyError: 2



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Set Operations

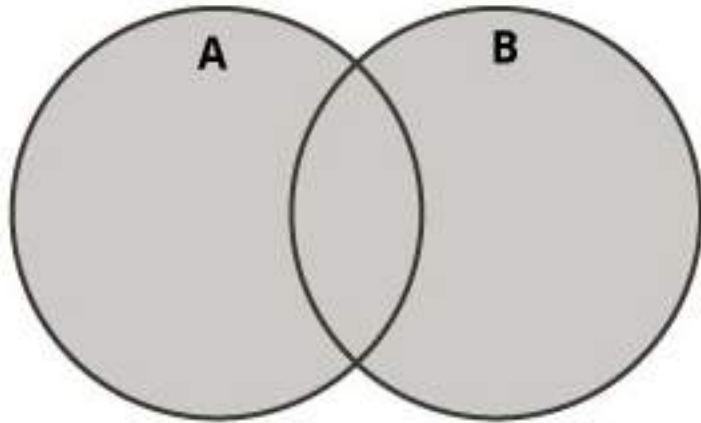
Sets can be used to carry out mathematical set operations like union, intersection, difference and symmetric difference. We can do this with operators or methods.

```
>>> A = {1, 2, 3, 4, 5}
```

```
>>> B = {4, 5, 6, 7, 8}
```

Set Union

U

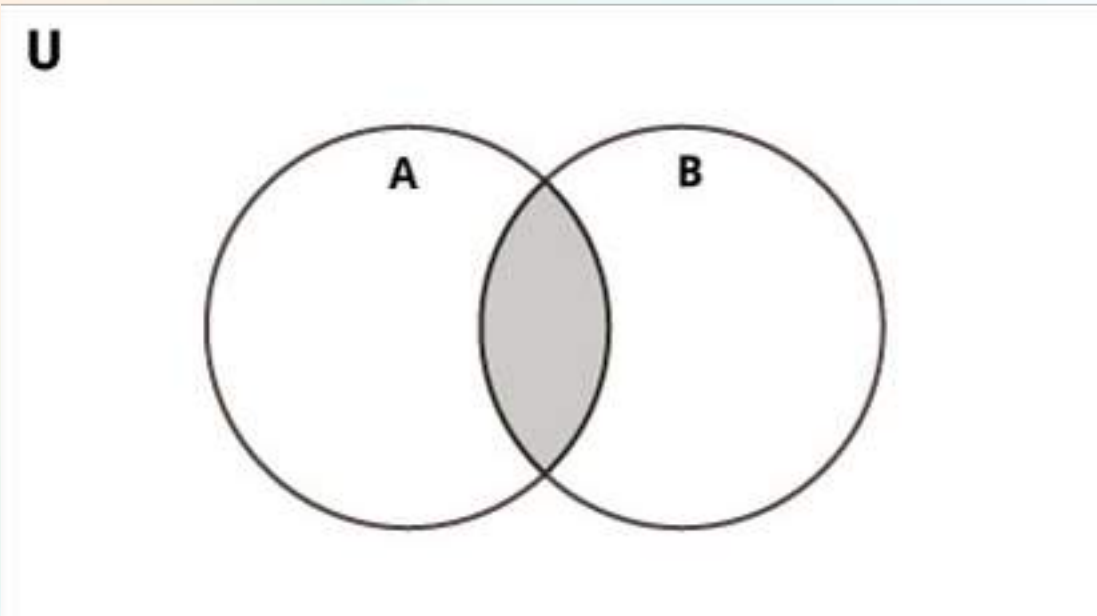


Union of A and B is a set of all elements from both sets. Union is performed using | operator. Same can be accomplished using the union() method

```
1 # Set union method
2 # initialize A and B
3 A = {1, 2, 3, 4, 5}
4 B = {4, 5, 6, 7, 8}
5
6 # use | operator
7 # Output: {1, 2, 3, 4, 5, 6, 7, 8}
8 print(A | B)
9
10 # use union function
11 print(A.union(B))
12
13 # use union function on B
14 print(B.union(A))
```

```
{1, 2, 3, 4, 5, 6, 7, 8}
{1, 2, 3, 4, 5, 6, 7, 8}
{1, 2, 3, 4, 5, 6, 7, 8}
```

Set Intersection



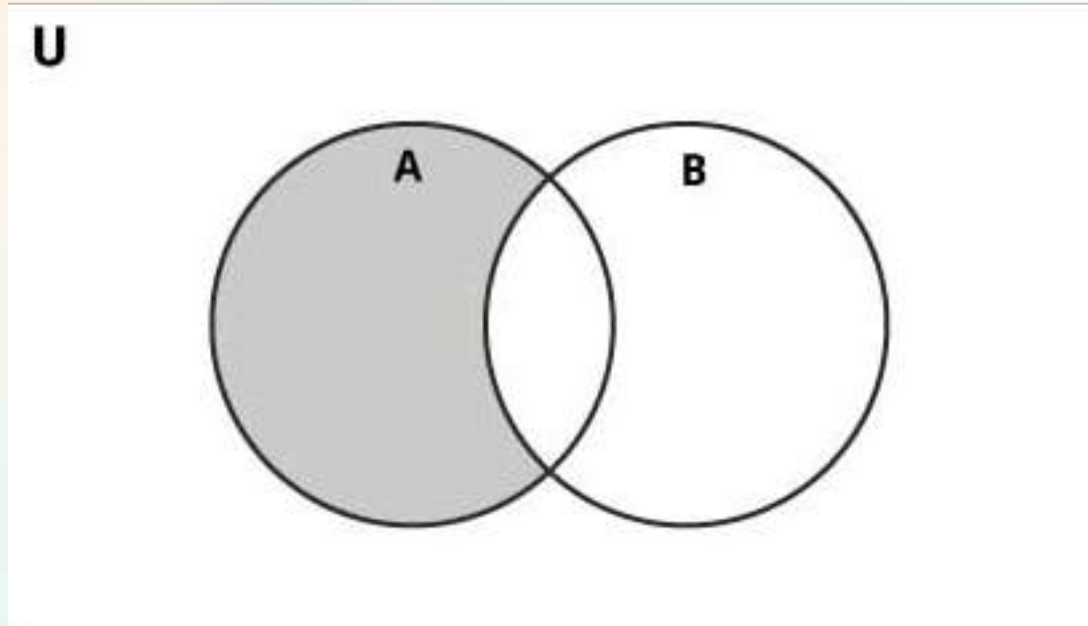
Intersection of A and B is a set of elements that are common in both the sets.

Intersection is performed using & operator. Same can be accomplished using the intersection() method.

```
1 # Intersection of sets
2 # initialize A and B
3 A = {1, 2, 3, 4, 5}
4 B = {4, 5, 6, 7, 8}
5 # use & operator
6 # Output: {4, 5}
7 print(A & B)
8 # use intersection function on A
9 print(A.intersection(B))
10
11 # use intersection function on B
12 print(B.intersection(A))
```

```
{4, 5}
{4, 5}
{4, 5}
```

Set Difference



Difference of the set B from set A ($A - B$) is a set of elements that are only in A but not in B. Similarly, $B - A$ is a set of elements in B but not in A.

Difference is performed using - operator. Same can be accomplished using the `difference()` method.

```
1 # Difference of two sets
2 # initialize A and B
3 A = {1, 2, 3, 4, 5}
4 B = {4, 5, 6, 7, 8}
5 # use - operator on A
6 # Output: {1, 2, 3}
7 print(A - B)
8 # use difference function on A
9 print(A.difference(B))
10 # use - operator on B
11 print(B - A)
12 # use difference function on B
13 print(B.difference(A))
```

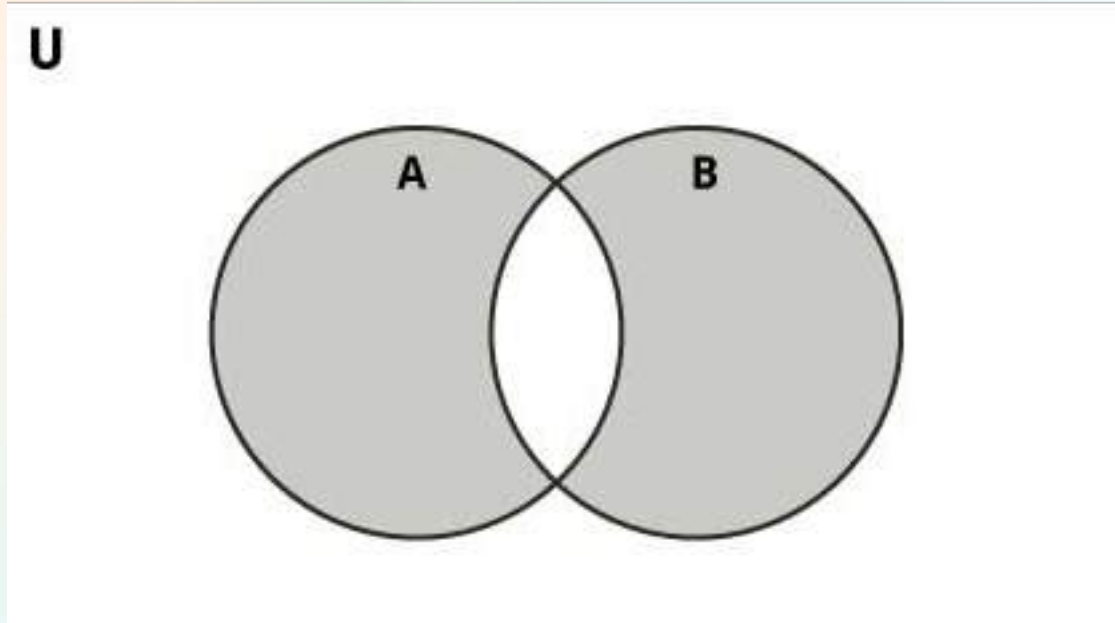
{1, 2, 3}

{1, 2, 3}

{8, 6, 7}

{8, 6, 7}

Set Symmetric Difference



Symmetric Difference of A and B is a set of elements in A and B but not in both (excluding the intersection). Symmetric difference is performed using ^ operator. Same can be accomplished using the method `symmetric_difference()`.

```
1 # Symmetric difference of two sets
2 # initialize A and B
3 A = {1, 2, 3, 4, 5}
4 B = {4, 5, 6, 7, 8}
5 # use ^ operator
6 # Output: {1, 2, 3, 6, 7, 8}
7 print(A ^ B)
8 # use symmetric_difference function on A
9 print(A.symmetric_difference(B))
10 # use symmetric_difference function on B
11 print(B.symmetric_difference(A))
```

```
{1, 2, 3, 6, 7, 8}
{1, 2, 3, 6, 7, 8}
{1, 2, 3, 6, 7, 8}
```

Other Python Set Methods

Method	Description
<code>add()</code>	Adds an element to the set
<code>clear()</code>	Removes all elements from the set
<code>copy()</code>	Returns a copy of the set
<code>difference()</code>	Returns the difference of two or more sets as a new set
<code>difference_update()</code>	Removes all elements of another set from this set
<code>discard()</code>	Removes an element from the set if it is a member. (Do nothing if the element is not in set)
<code>intersection()</code>	Returns the intersection of two sets as a new set
<code>intersection_update()</code>	Updates the set with the intersection of itself and another

<code>isdisjoint()</code>	Returns <code>True</code> if two sets have a null intersection
<code>issubset()</code>	Returns <code>True</code> if another set contains this set
<code>issuperset()</code>	Returns <code>True</code> if this set contains another set.
<code>pop()</code>	Removes and returns an arbitrary set element. Raises <code>KeyError</code> if the set is empty
<code>remove()</code>	Removes an element from the set. If the element is not a member, raises a <code>KeyError</code>
<code>symmetric_difference()</code>	Returns the symmetric difference of two sets as a new set
<code>symmetric_difference_update()</code>	Updates a set with the symmetric difference of itself and another
<code>union()</code>	Returns the union of sets in a new set
<code>update()</code>	Updates the set with the union of itself and others.

Built-in Functions with Set

Built-in functions like `all()`, `any()`, `enumerate()`, `len()`, `max()`, `min()`, `sorted()`, `sum()` etc. are commonly used with sets to perform different tasks.

Function	Description
<code>all()</code>	Returns <code>True</code> if all elements of the set are true (or if the set is empty).
<code>any()</code>	Returns <code>True</code> if any element of the set is true. If the set is empty, returns <code>False</code> .
<code>enumerate()</code>	Returns an enumerate object. It contains the index and value for all the items of the set as a pair.
<code>len()</code>	Returns the length (the number of items) in the set.
<code>max()</code>	Returns the largest item in the set.
<code>min()</code>	Returns the smallest item in the set.
<code>sorted()</code>	Returns a new sorted list from elements in the set(does not sort the set itself).
<code>sum()</code>	Returns the sum of all elements in the set.

Python Frozenset

Frozenset is a new class that has the characteristics of a set, but its elements cannot be changed once assigned. While tuples are immutable lists, frozensets are immutable sets.

Sets being mutable are unhashable, so they can't be used as dictionary keys. On the other hand, frozensets are hashable and can be used as keys to a dictionary.

Frozensets can be created using the `frozenset()` function.

This data type supports methods like `copy()`, `difference()`, `intersection()`, `isdisjoint()`, `issubset()`, `issuperset()`, `symmetric_difference()` and `union()`. Being immutable, it does not have methods that add or remove elements.

Python Dictionary

Python Dictionary

Python dictionary is an unordered collection of items. Each item of a dictionary has a key/value pair.

Dictionaries are optimized to retrieve values when the key is known.

Creating Python Dictionary

Creating a dictionary is as simple as placing items inside curly braces {} separated by commas.

An item has a key and a corresponding value that is expressed as a pair (**key: value**).

While the values can be of any data type and can repeat, keys must be of immutable type (string, number or tuple with immutable elements) and must be unique.

Example

```
1 # empty dictionary
2 my_dict = {}
3 # dictionary with integer keys
4 my_dict = {1: 'apple', 2: 'ball'}
5 print(my_dict)
6 # dictionary with mixed keys
7 my_dict = {'name': 'John', 1: [2, 4, 3]}
8 print(my_dict)
9 # using dict()
10 my_dict = dict({1: 'apple', 2: 'ball'})
11 print(my_dict)
12 # from sequence having each item as a pair
13 my_dict = dict([(1, 'apple'), (2, 'ball')])
14 print(my_dict)
```

```
{1: 'apple', 2: 'ball'}
{'name': 'John', 1: [2, 4, 3]}
{1: 'apple', 2: 'ball'}
{1: 'apple', 2: 'ball'}
```

Accessing Elements from Dictionary

While indexing is used with other data types to access values, a dictionary uses keys. Keys can be used either inside square brackets [] or with the get() method.

If we use the square brackets [], KeyError is raised in case a key is not found in the dictionary. On the other hand, the get() method returns None if the key is not found.

Example

```
1 # get vs [] for retrieving elements
2 my_dict = {'name': 'Ivana', 'age': 18}
3
4 # Output: Jack
5 print(my_dict['name'])
6
7 # Output: 26
8 print(my_dict.get('age'))
9
10 # Trying to access keys which doesn't exist throws error
11 # Output None
12 print(my_dict.get('address'))
13
14 # KeyError
15 print(my_dict['address'])
```

Ivana

18

None

Traceback (most recent call last):

File "Untitled 7.py", line 15, in <module>

print(my_dict['address'])

KeyError: 'address'

Changing and Adding Dictionary elements

Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.

If the key is already present, then the existing value gets updated. In case the key is not present, a new (**key: value**) pair is added to the dictionary.

Example

```
1 # Changing and adding Dictionary Elements
2 my_dict = {'name': 'Ivana', 'age': 18}
3
4 # update value
5 my_dict['age'] = 27
6
7 print(my_dict)
8
9 # add item
10 my_dict['address'] = 'Downtown'
11
12 print(my_dict)
```

```
{'name': 'Ivana', 'age': 27}
```

```
{'name': 'Ivana', 'age': 27, 'address': 'Downtown'}
```

Removing elements from Dictionary

We can remove a particular item in a dictionary by using the `pop()` method. This method removes an item with the provided key and returns the value.

The `popitem()` method can be used to remove and return an arbitrary (key, value) item pair from the dictionary. All the items can be removed at once, using the `clear()` method.

We can also use the `del` keyword to remove individual items or the entire dictionary itself.

Example

```
1 # create a dictionary
2 squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
3 # remove a particular item, returns its value
4 # Output: 16
5 print(squares.pop(4))
6 # Output: {1: 1, 2: 4, 3: 9, 5: 25}
7 print(squares)
8 # remove an arbitrary item, return (key,value)
9 # Output: (5, 25)
10 print(squares.popitem())
11 # Output: {1: 1, 2: 4, 3: 9}
12 print(squares)
13 # remove all items
14 squares.clear()
15 # Output: {}
16 print(squares)
17 # delete the dictionary itself
18 del squares
19 # Throws Error
20 print(squares)
```

```
16
{1: 1, 2: 4, 3: 9, 5: 25}
(5, 25)
{1: 1, 2: 4, 3: 9}
{}
Traceback (most recent call last):
  File "Untitled 7.py", line 20, in <module>
    print(squares)
NameError: name 'squares' is not defined
```


Python Dictionary Methods

Method	Description
<code>clear()</code>	Removes all items from the dictionary.
<code>copy()</code>	Returns a shallow copy of the dictionary.
<code>fromkeys(seq[, v])</code>	Returns a new dictionary with keys from <code>seq</code> and value equal to <code>v</code> (defaults to <code>None</code>).
<code>get(key[,d])</code>	Returns the value of the <code>key</code> . If the <code>key</code> does not exist, returns <code>d</code> (defaults to <code>None</code>).
<code>items()</code>	Return a new object of the dictionary's items in (key, value) format.
<code>keys()</code>	Returns a new object of the dictionary's keys.
<code>pop(key[,d])</code>	Removes the item with the <code>key</code> and returns its value or <code>d</code> if <code>key</code> is not found. If <code>d</code> is not provided and the <code>key</code> is not found, it raises <code>KeyError</code> .

<code>popitem()</code>	Removes and returns an arbitrary item (key, value). Raises <code>KeyError</code> if the dictionary is empty.
<code>setdefault(key[,d])</code>	Returns the corresponding value if the <code>key</code> is in the dictionary. If not, inserts the <code>key</code> with a value of <code>d</code> and returns <code>d</code> (defaults to <code>None</code>).
<code>update[other]</code>	Updates the dictionary with the key/value pairs from <code>other</code> , overwriting existing keys.
<code>values()</code>	Returns a new object of the dictionary's values.

Example

```
1 # Dictionary Methods
2 marks = {}.fromkeys(['Math', 'English', 'Science'], 0)
3
4 # Output: {'English': 0, 'Math': 0, 'Science': 0}
5 print(marks)
6
7 for item in marks.items():
8     print(item)
9
10 # Output: ['English', 'Math', 'Science']
11 print(list(sorted(marks.keys())))
```

```
{'Math': 0, 'English': 0, 'Science': 0}
('Math', 0)
('English', 0)
('Science', 0)
['English', 'Math', 'Science']
```


Python Dictionary Comprehension

Dictionary comprehension is an elegant and concise way to create a new dictionary from an iterable in Python.

Dictionary comprehension consists of an expression pair (**key: value**) followed by a for statement inside curly braces {}.

Example

```
1 # Dictionary Comprehension
2 squares = {x: x*x for x in range(6)}
3
4 print(squares)
5
6 #Code equivalent
7 squares = {}
8 for x in range(6):
9     squares[x] = x*x
10 print(squares)
11
12 # Dictionary Comprehension with if conditional
13 odd_squares = {x: x*x for x in range(11) if x % 2 == 1}
14
15 print(odd_squares)
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
```

A dictionary comprehension can optionally contain more **for** or **if** statements.

An optional if statement can filter out items to form the new dictionary.

Here are some examples to make a dictionary with only odd items.



Dictionary Built-in Functions

Built-in functions like `all()`, `any()`, `len()`, `cmp()`, `sorted()`, etc. are commonly used with dictionaries to perform different tasks.

Function	Description
<code>all()</code>	Return <code>True</code> if all keys of the dictionary are <code>True</code> (or if the dictionary is empty).
<code>any()</code>	Return <code>True</code> if any key of the dictionary is true. If the dictionary is empty, return <code>False</code> .
<code>len()</code>	Return the length (the number of items) in the dictionary.
<code>cmp()</code>	Compares items of two dictionaries. (Not available in Python 3)
<code>sorted()</code>	Return a new sorted list of keys in the dictionary.

```
1 # Dictionary Built-in Functions
2 squares = {0: 0, 1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
3 # Output: False
4 print(all(squares))
5 # Output: True
6 print(any(squares))
7 # Output: 6
8 print(len(squares))
9 # Output: [0, 1, 3, 5, 7, 9]
10 print(sorted(squares))
```

```
False
True
6
[0, 1, 3, 5, 7, 9]
```

